CS 630: Software Testing and Quality Assurance

4-1 Activity: Applying TDD With Pytest

Malgorzata Debska

Southern New Hampshire University

Dr. Ben Torrales

July 28, 2026

Applying Test-Driven Development with Pytest

For this activity, I followed the test-driven development (TDD) approach by implementing the red-green-refactor cycle throughout the assignment. Instead of writing the implementation first, I began identifying behaviors that were missing from the existing test suite. The original project only verified that the Roman numeral "I" returned the value 1, leaving many required behaviors untested. I identified additional functionality that needed to be verified, including additive Roman numerals, subtractive notation, larger values, invalid inputs, and boundary cases.

The first step of the TDD cycle was the red phase. During this phase, I created new Pytest unit tests for expected behaviors before modifying the application code. I wrote tests for values such as II, III, IV, IX, LVIII, and MCMXCIV, along with tests for invalid inputs such as empty strings, invalid Roman numeral characters, and incorrectly formatted numerals. As expected, these tests initially failed because the provided implementation always returned the value 1regardless of the input.

The next step was the green phase. After confirming that the new tests failed, I modified the Python implementation to correctly convert Roman numerals into integers. I implemented logic that recognizes both additive notations, where values are added together, and subtractive notation, where a smaller numeral placed before a larger numeral represents subtraction. I also added input validation to reject invalid Roman numerals and ensure that only properly formatted values are accepted. After making these changes, I ran pytest again until every test passed successfully.

The final phase was refactoring. Once all tests passed, I reviewed the code to improve readability and maintainability without changing its behavior. I simplified sections of the conversion logic, organized validation into clear steps, removed unnecessary code, and used meaningful variable names. Because the complete test suite continued to pass after each change, I was confident that the refactoring did not introduce any new defects.

My test design supports requirement verification because it covers both normal and exceptional scenarios. The tests verify common Roman numerals, subtractive combinations, larger values, and invalid input cases. Testing edge cases is especially important because they often reveal defects that are not detected through simple examples. By validating both expected outputs and error conditions, the test suite provides confidence that the implementation behaves correctly under different situations.

One challenge I encountered was handling the rules for Roman numeral validation. While converting simple numerals is straightforward, correctly recognizing subtractive notation and rejecting invalid combinations requires additional logic. I addressed this challenge by implementing validation before performing the conversion and then repeatedly running the test suite after each change. Using the TDD process made debugging much easier because each failing test clearly identified which requirement still needed to be implemented.

Overall, this activity demonstrated the value of test-driven development. Writing tests first encouraged me to think carefully about the expected behavior before writing the implementation. The repeated red-green-refactor cycle helped produce reliable, maintainable code while ensuring that every requirement was verified through automated testing.